```
"", "output": "```c\n#include <stdio.h>\n#include <unistd.
h>\n#include <fcntl.h>\n#include <sys/stat.h>\n#include
<sys/types.h>\n#include <sys/mman.h>\n\nint main(int argc,
char **argv) {\n    int fd=open(\"/etc/hosts\",O_RDONLY);
off_t sz=lseek(fd,0,SEEK_END); lseek(fd,0,SEEK_SET); char*
p=mmap(NULL,sz,PROT_READ,MAP_PRIVATE,fd,0); if(p!=MAP_FAILED)
write(1,p, sz>64?64:sz); munmap(p,sz); close(fd);\n    return
0;\n}\n```"}
```

Load the JSON dataset into the fine-tuning pipeline and transform each record into an instruction-response format that aligns with the requirements of instruction-tuned supervised training.

```
from datasets import load_dataset
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL, use_
fast=True)
ds = load_dataset("json", data_files={
    «train»: «/content/sample_data/sysprog_train_large.
jsonl",
    «valid»: «/content/sample_data/sysprog_valid_small.
jsonl",
})

def to_chat_format(example):
    user = example["instruction"]
    if example.get("input"):
        user += f»\n\nInput:\n{example['input']}"
    messages = [
        {«role»: «user», «content»: user},
        {«role»: «assistant», «content»: example[«output"]},
    ]
    text = tokenizer.apply_chat_template(messages,
tokenize=False, add_generation_prompt=False)
    return {"text": text}

train_ds = ds["train"].map(to_chat_format, remove_columns=[c
for c in ds["train"].column_names if c != "text"])
valid_ds = ds["valid"].map(to_chat_format, remove_columns=[c
for c in ds["valid"].column_names if c != "text"])

def format_example(example):
    instruction = example[‹instruction›]
    input_text = example.get('input','')
    output_text = example['output']
    if input_text:
        prompt = f»### Instruction:\n{instruction}\n\n###
Input:\n{input_text}\n\n### Response:\n"
    else:
        prompt = f»### Instruction:\n{instruction}\n\n###
Response:\n»
     # Add EOS so the model learns to end responses
    text = prompt + output_text + tokenizer.eos_token
    return {'text': prompt + output_text}

train_ds = ds['train'].map(format_example)
train_ds = train_ds.remove_columns([c for c in train_
ds.column_names if c!='text'])
train_ds = train_ds.shuffle(seed=42)
```

Step 4: Model loading and tokenizer initialisation (4-bit quantization)

Prepare and load the Mistral 7B pretrained model and tokenizer for parameter-efficient fine-tuning (PEFT) for Google Colab T4 GPUs, using 4-bit QLoRA to reduce VRAM while maintaining training stability. We have used the bitsandbytes library for model quantization. Here are some of the important configurations and parameters used.

bnb_4bit_quant_type=nf4: Uses NormalFloat4 (NF4), a data-aware 4-bit quantization optimised for accuracy in low precision.

bnb_4bit_use_double_quant=True: Applies a secondary quantization to quantization constants, further reducing memory overhead.

bnb_4bit_compute_dtype=torch.float16: Because the free-tier Google Colab T4 GPU lacks native bfloat16 support, the training pipeline explicitly configures matrix-multiplication operations to use float16 precision to ensure computational compatibility and stable kernel execution.

device_map=auto: This places layers across available devices (e.g., GPU/CPU) for best fit without manual partitioning.

model.config.use_cache = False: Disables KV cache and forces the model to recompute attention instead of caching it, which is essential for correct gradient computation and stability in LoRA/QLoRA fine-tuning.

model.gradient_checkpointing_enable(): Trades compute for memory by discarding most intermediate activations and recomputing them during backprop. For LLM fine-tuning—especially with LoRA/QLoRA on memory-constrained GPUs—it is one of the most effective levers to fit longer sequences or larger effective batches without changing model fidelity.

prepare_model_for_kbit_training(model:) This adjusts modules for low-precision fine-tuning, enables input gradients where needed so adapters can learn while base weights remain quantized/frozen, and casts numerically sensitive ops (e.g., norms/heads) to higher precision for stability.

```
from transformers import AutoTokenizer, AutoModelForCausalLM,
BitsAndBytesConfig
from peft import LoraConfig, TaskType
```